

Lista de Exercícios — Sistemas de Arquivos — Respostas

1.

	XU-A	XU-B
(a)	1000 setores	1110 setores
(b)	500 setores	15 setores
2.

diretos = $12 \times 4 \text{ KB} = 48 \text{ KB}$
 indiretos = $2 \times (4 \text{ KB}/4) \times 4 \text{ KB} = 8 \text{ MB}$
 duplamente indiretos = $2 \times (4 \text{ KB}/4) \times (4 \text{ KB}/4) \times 4 \text{ KB} = 8 \text{ GB}$
 tamanho máximo = $8 \text{ GB} + 8 \text{ MB} + 48 \text{ KB} = 8.396.848 \text{ KB}$
3. Supõe-se aqui que blocos lógicos e físicos têm o mesmo tamanho.
 - (a) $5 \text{ KB (dir.)} + 256 \text{ KB (indir.)} + 65.536 \text{ KB (dupl. indir.)} = 65.797 \text{ KB}$
 - (b) 515 blocos
4. O sistema de arquivos ocupa 20 GB, o que corresponde a

$$\frac{20 \text{ GB}}{512} = \frac{20 \cdot 2^{30}}{2^9} = 20 \cdot 2^{21} \text{ blocos de 512 bytes}$$

- (a) Caso seja usada uma lista de blocos livres, quando o sistema de arquivos estiver vazio, todos os seus $20 \cdot 2^{21}$ blocos estarão livres. Cada bloco de 512 bytes pode armazenar $512/4 = 128 (2^7)$ endereços de bloco. Portanto, a lista de livres ocupará $\frac{20 \cdot 2^{21}}{2^7} = 20 \cdot 2^{14}$ blocos. Como cada bloco possui 512 bytes, o espaço ocupado será de

$$20 \cdot 2^{14} \times 512 = 167.772.160 \text{ bytes} = 163.840 \text{ KB} = 160 \text{ MB}$$

- (b) O mapa de bits possui tamanho fixo, independente dos blocos estarem livres ou ocupados. Com $20 \cdot 2^{21}$ blocos, são necessários $20 \cdot 2^{21}$ bits, o que corresponde a $\frac{20 \cdot 2^{21}}{8} = 5.242.880$ bytes = 5.120 KB = 5 MB.

5. Para abrir um arquivo, é necessário ler o seu inode do disco para a memória. O diretório raiz (na memória) contém o número do inode de /usr. Partindo desse número, são realizadas as seguintes operações em disco:

1	leitura do inode de usr	encontra o bloco com o conteúdo do diretório /usr
2	leitura do conteúdo de usr	descobre o número do inode de bin
3	leitura do inode de bin	encontra o bloco com o conteúdo do diretório /usr/bin
4	leitura do conteúdo de bin	descobre o número do inode de finger
5	leitura do inode de finger	
5 leituras no disco		

Conforme definido no enunciado, nos passos 2 e 4 supõe-se que as entradas bin e finger estejam contidas no primeiro bloco dos respectivos diretórios.

6. Considerando um inode com 10 endereços diretos, um ponteiro de indireção simples, um ponteiro de indireção dupla e um ponteiro de indireção tripla, além de endereços de disco de 4 bytes, o arquivo ocupa $\lceil \frac{20.000.000}{1024} \rceil = 19.532$ blocos de dados. Desses, 10 blocos são diretos (apontados no próprio inode). Dos 19.522 blocos restantes, $1024/4 = 256$ blocos são apontados pelo bloco de indireção simples.
 Os $19.522 - 256 = 19.266$ blocos restantes estão contidos nos $(1024/4) \times (1024/4) = 65.536$ blocos apontados por indireção dupla; os blocos de dados requerem $\lceil \frac{19.266}{256} \rceil = 76$ blocos de índices.
 Portanto, no total o arquivo ocupa

	19.532	blocos de dados
+	1	bloco para o inode
+	1	bloco de índices (indireção simples)
+	77	blocos de índices (indireção dupla)
	19.611	blocos

Neste exercício o inode é contado, pois se está falando de espaço ocupado, não de blocos lidos.

7. (a) 1111 1111 1111 0000
 (b) 1000 0001 1111 0000
 (c) 1111 1111 1111 1100
 (d) 1111 1110 0000 1100
8. (a) É possível apontar a existência de 3 arquivos:
 - A1: 13, 2, 9, 7, 12
 - A2: 11, 8, 3, 14
 - A3: 15, 10, 5
 (b) 0011 0101 1111 1111
 (c) O mapa de bits aponta 4 blocos livres. O espaço livre disponível é, então, $4 \times 4 = 16$ KB.
 (d) A FAT tem 16 entradas com, no mínimo, 5 bits por entrada (4 bits para o número do bloco, de 0 a 15, e 1 bit para sinal, para representar o ponteiro nulo). Portanto, ela requer pelo menos $16 \times 5 = 80$ bits, ou 10 bytes.
9. (a) blocos diretos: $11 \times 4 \text{ KB} = 44 \text{ KB}$
 indireção simples: $1024 \times 4 \text{ KB} = 4096 \text{ KB}$
 indireção dupla: $2 \times 1024 \times 1024 \times 4 \text{ KB} = 8 \text{ GB}$
 tam. máximo: $44 \text{ KB} + 4096 \text{ KB} + 8 \text{ GB} = 8.392.748 \text{ KB}$
 (b) O aumento será de $11 \times 4 \text{ KB} = 44 \text{ KB}$.
 (c) 2501 blocos de dados + 4 blocos de índices = 2505 blocos

10. (1) /snoopy 2 blocos + 1 descritor
- conteúdo do diretório / 1 bloco
 - descritor do arquivo /snoopy 1 descritor
 - conteúdo do arquivo /snoopy 1 bloco
- (2) /gotham/batman 2 blocos + 2 descritores
- conteúdo do diretório / em *cache*
 - descritor do diretório /gotham 1 descritor
 - conteúdo do diretório /gotham 1 bloco
 - descritor do arquivo /gotham/batman 1 descritor
 - conteúdo do arquivo /gotham/batman 1 bloco
- (3) /gotham/robin 2 blocos + 1 descritor
- conteúdo do diretório / em *cache*
 - descritor do diretório /gotham em *cache*
 - conteúdo do diretório /gotham em *cache*
 - descritor do arquivo /gotham/robin 1 descritor
 - conteúdo do arquivo /gotham/robin 2 blocos
- (4) /gotham/batman 3 blocos + 2 descritores
- conteúdo do diretório / em *cache*
 - descritor do diretório /metropolis 1 descritor
 - conteúdo do diretório /metropolis 1 bloco
 - descritor do arquivo /metropolis/superman 1 descritor
 - conteúdo do arquivo /metropolis/superman 2 blocos
- (5) /gotham/arkham/joker 4 blocos + 2 descritores
- conteúdo do diretório / em *cache*
 - descritor do diretório /gotham em *cache*
 - conteúdo do diretório /gotham em *cache*
 - descritor do diretório /gotham/arkham 1 descritor
 - conteúdo do diretório /gotham/arkham 1 bloco
 - descritor do arquivo /gotham/arkham/joker 1 descritor
 - conteúdo do arquivo /gotham/arkham/joker 3 blocos
- (6) /garfield 3 blocos + 1 descritor
- conteúdo do diretório / em *cache*
 - descritor do arquivo /garfield 1 descritor
 - conteúdo do arquivo /garfield 3 blocos

Portanto, devem ser lidos 16 blocos (sendo 12 para dados e 4 para diretórios), o que corresponde a $16 \times 8 = 128$ setores. Como devem ser lidos ainda 9 descritores e nenhum par de descritores compartilha o mesmo setor (considerações ii e iii), será preciso ler 9 setores adicionais, chegando a um total de $128 + 9 = 137$ setores.

11. O raciocínio abaixo considera que o arquivo atual contém 200 blocos lógicos, numerados de 0 a 199. Na alocação contígua, todas as operações exigem a atualização do diretório para incrementar/decrementar o número de blocos do arquivo, mas o enunciado define (na consideração i) que isso é feito em memória. Os casos em que é necessário atualizar o diretório na alocação encadeada são notados.
- (a) **contígua** como só existe espaço para expansão no final, é necessário deslocar todos os blocos (200 leituras, 200 escritas) para o final do arquivo e escrever o novo bloco (1 escrita).
- encadeada** basta escrever o novo bloco, fazendo com que o seu próximo bloco seja o atual bloco 0 (0 leituras, 1 escrita), e atualizar o primeiro bloco na entrada de diretório (em memória, 0 leituras/escritas).
- indexada** é preciso gravar o novo bloco (1 escrita), ler o inode e atualizar a lista de blocos do arquivo (1 leitura, 1 escrita).
- (b) **contígua** é preciso deslocar os blocos 100–199 uma posição para o final (100 leituras, 100 escritas), e gravar o novo bloco (1 escrita).
- encadeada** é preciso ler os blocos 0–99 e atualizar o próximo do bloco 99 (100 leituras, 1 escrita), além de gravar o novo bloco (1 escrita).
- indexada** é preciso gravar o novo bloco (1 escrita), ler o inode e atualizar a lista de blocos do arquivo (1 leitura, 1 escrita).
- (c) **contígua** é preciso apenas gravar o novo bloco (1 escrita).
- encadeada** é preciso ler os blocos 0–199 e atualizar o próximo do bloco 199 (200 leituras, 1 escrita), além de gravar o novo bloco (1 escrita).
- indexada** é preciso gravar o novo bloco (1 escrita), ler o inode e atualizar a lista de blocos do arquivo (1 leitura, 1 escrita).
- (d) **contígua** supondo que o bloco 0 possa ser liberado, é necessário alterar apenas as estruturas de dados em memória (o diretório, para incrementar o bloco inicial, e a lista/mapa de blocos livres), sem leituras ou escritas de blocos de dados ou índices.
- encadeada** é preciso apenas ler o bloco 0 para obter o endereço do bloco 1, que será registrado no diretório como novo bloco inicial do arquivo (1 leitura, 0 escritas).
- indexada** é preciso ler o inode e atualizar a lista de blocos do arquivo (1 leitura, 1 escrita).
- (e) Supondo que seja removido o bloco 100:
- contígua** é preciso deslocar os blocos 101–199 uma posição para o início do arquivo (99 leituras, 99 escritas).
- encadeada** é preciso ler os blocos 0–100 (para obter o endereço do bloco 101) e atualizar o próximo do bloco 99 (101 leituras, 1 escrita).
- indexada** é preciso ler o inode e atualizar a lista de blocos do arquivo (1 leitura, 1 escrita).
- (f) **contígua** supondo que o bloco 199 possa ser liberado, é necessário alterar apenas as estruturas de dados em memória (o diretório, para decrementar a contagem de blocos, e a lista/mapa de blocos livres), sem leituras ou escritas de blocos de dados ou índices.
- encadeada** é preciso apenas ler os blocos 0–198 e atualizar o próximo do bloco 198 (199 leituras, 1 escrita).
- indexada** é preciso ler o inode e atualizar a lista de blocos do arquivo (1 leitura, 1 escrita).

Resumo:

operação	alocação contígua		alocação encadeada		alocação indexada	
	leituras	escritas	leituras	escritas	leituras	escritas
(a)	200	201	0	1	1	2
(b)	100	101	100	2	1	2
(c)	0	1	200	2	1	2
(d)	0	0	1	0	1	1
(e)	99	99	101	1	1	1
(f)	0	0	199	1	1	1

12. Cada bloco de 2 KB ocupa 4 setores de 512 bytes. Como o setor inicial é 102400, tem-se:

bloco	setor inicial	setor final
0	102400	102403
1	102404	102407
2	102408	102411
$\vdots$	$\vdots$	$\vdots$
k	$102400 + 4k$	$102400 + 4(k + 1) - 1$

Serão lidos três blocos, de 171 a 173. O bloco 171 inicia no setor $102400 + (4 \times 171) = 103084$, e o bloco 173 termina no setor $102400 + (4 \times 174) - 1 = 103095$. Portanto, serão lidos 12 setores, de 103084 a 103095.